

Week11_예습과제_이재린

≡ 태그

Ch6. 임베딩

Word2Vec

Word2Vec란

- why Word2Vec? 빈도 기반의 vectorize에서 저장하지 못하는 단어의 의미 정보를 저장 가능함 & 대량의 텍스트 데이터에서 단어 관계 파악 + 공간상 유사한 단어를 군집화할 수 있음
 - Word2Vec : 분포 가설을 기반으로 각 단어의 유사성(단어간의 동시 발생 확률 분포)을 측정
 - 분포 가설 : 같은 문맥에서 자주 함께 나타나는 단어는 서로 유사한 의미를 가질 가능성이 높다는 가설 → 단어간의 동시 발생 확률 분포로 단어간의 유사성 측정
- ex) 나는 사과를 먹는다 & 나는 치킨을 먹는다.
- 사과와 치킨 주변에 나타나는 단어들이 동일하거나 유사하므로, 둘은 의미가 비슷할 것이라 예상
- 이러한 가설로 분산 표현 학습
- 분산 표현 : 단어를 고차원 벡터 공간에 매핑하여 단어의 의미를 담는 것
- 사과와 치킨은 벡터 공간상에서 서로 가까운 위치에 표현됨.

단어 벡터화

하는 방법은 두가지로 나뉘는데...

1. 희소 표현

- 원-핫 인코딩, TF-IDF 등 빈도 기반 방법
- 대부분의 0 >> 단어 간의 유사성 반영 X, 계산 비용 up
- 단어 사전 커지면 벡터의 크기 up

표 6.3 단어의 희소 표현

소	0	1	0	0	0
읽고	1	0	0	0	0
외양간	0	0	0	1	0
고친다	0	0	0	0	1

2. 밀집 표현

- 단어를 고정된 크기의 실수 벡터로 >> 공간 효율적 사용
- 벡터 공간 거리 O >> 학습을 통해 단어 벡터화 >> 단어 간 의미 비교
- 밀집 표현된 벡터 = 단어 임베딩 벡터

표 6.4 단어의 밀집 표현

소	0.3914	-0.1749	...	0.5912	0.1321
읽고	-0.2893	0.3814	...	-0.1492	-0.2814
외양간	0.4812	0.1214	...	-0.2745	0.0132
고친다	-0.1314	-0.2809	...	0.2014	0.3016

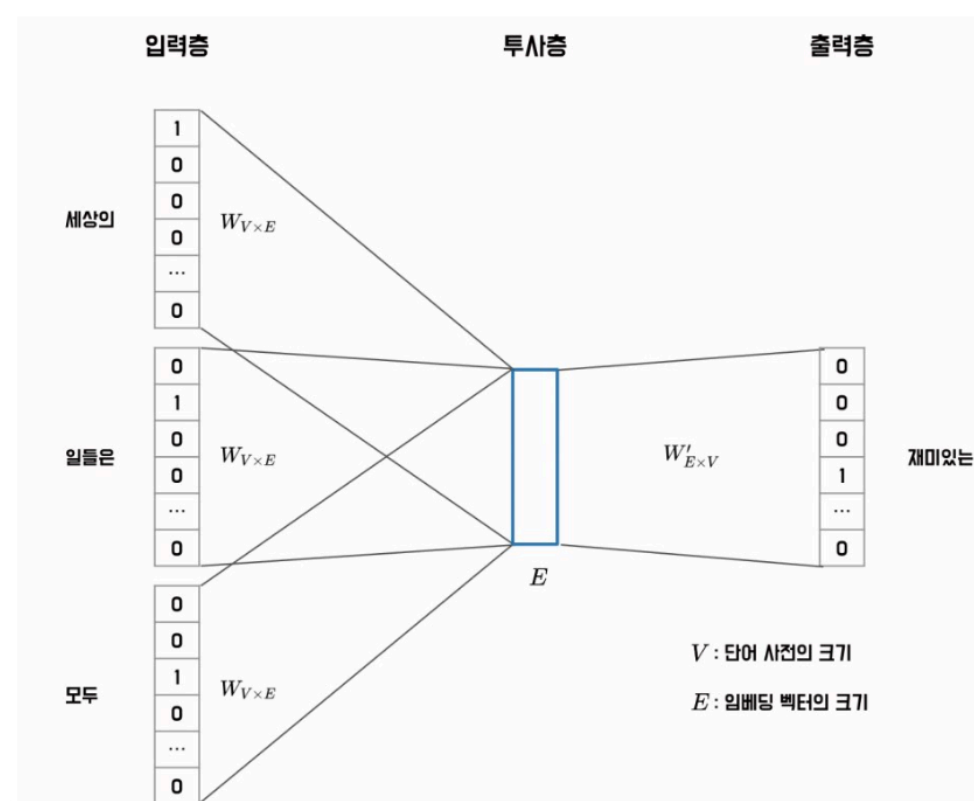
→ Word2Vec는 밀집 표현으로 CBoW와 Skip-gram 두 가지를 사용

CBoW

- CBoW : 주변 단어(context words)을 보고 중심 단어(center word)를 예측하는 방법
 - 윈도우(Window) : 중심 단어를 맞출 때 고려할 주변 단어의 개수
 - 슬라이딩 윈도우 : 문장 전체에서 중심단어를 바꿔 가며 학습하는 방식(앞 뒤로 윈도우 수만큼의 주변 단어 지정)
- 학습 데이터 구성 : (주변 단어 | 중심 단어)

입력 문장	학습 데이터
(세상의 재미있는 일들은)모두 밤에 일어난다	(재미있는, 일들은 세상의)
(세상의 재미있는 일들은 모두)밤에 일어난다	(세상의, 일들은, 모두 재미있는)
(세상의 재미있는 일들은 모두 밤에)일어난다	(세상의, 재미있는, 모두, 밤에 일들은)
세상의(재미있는 일들은 모두 밤에 일어난다)	(재미있는, 일들은, 밤에, 일어난다 모두)
세상의 재미있는(일들은 모두 밤에 일어난다)	(일들은, 모두, 일어난다 밤에)
세상의 재미있는 일들은(모두 밤에 일어난다)	(모두, 밤에 일어난다)

- CBoW의 모델구조



- (1) 입력층 : 각 입력 단어의 원-핫 벡터로 변환
- (2) 투사층 : 원-핫 벡터를 임베딩 벡터로 변환하는 Lookup table로 구성 → 크기가 E 인 임베딩 벡터로 변환

(3) 출력층 : 평균 벡터와 가중치 행렬($E \times V$)을 곱해 차원 V 크기의 출력 벡터를 얻음 → 소프트맥스 활성화로 중심 단어 예측

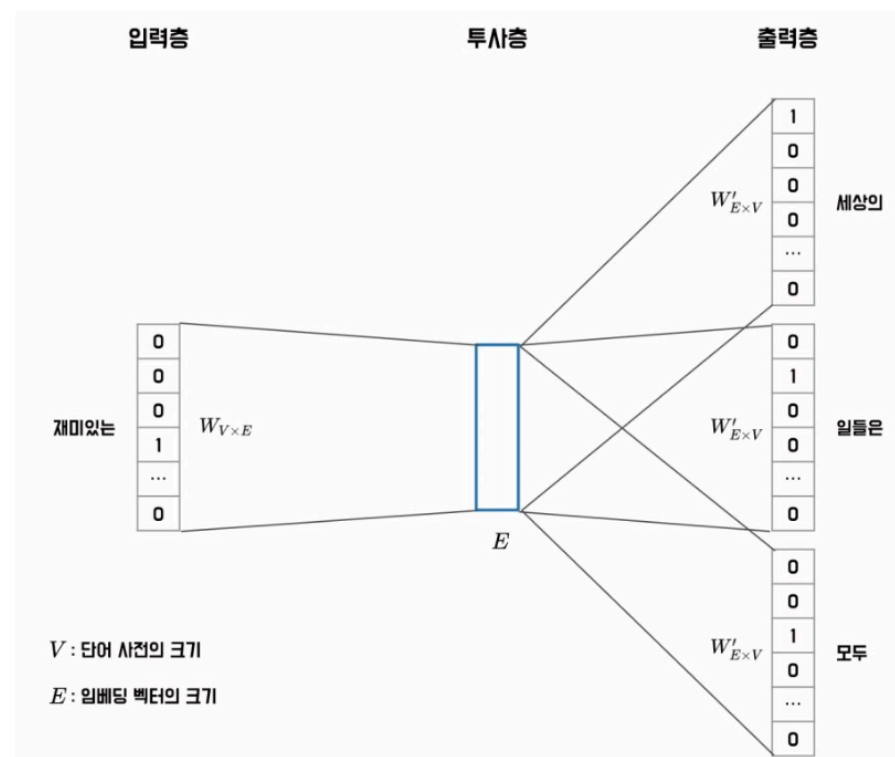
Skip-gram

- Skip-gram : CBoW와 반대로 중심 단어를 입력받아, 그 주변의 문맥 단어를 예측하는 모델
 - Skip-gram과 CBoW와 다른 점 : 중심 단어와 주변 단어를 하나의 쌍으로 하여 여러 학습 데이터를 생성함

입력 문장	학습 데이터
(세상의 재미있는 일들은)모두 밤에 일어난다	(세상의 재미있는), (세상의 일들은)
(세상의 재미있는 일들은 모두)밤에 일어난다	(재미있는 세상의), (재미있는 일들은), (재미있는 모두)
(세상의 재미있는 일들은 모두 밤에)일어난다	(일들은 세상의), (일들은 재미있는), (일들은 모두), (일들은 밤에)
세상의(재미있는 일들은 모두 밤에 일어난다)	(모두 재미있는), (모두 일들은), (모두 밤에), (모두 일어난다)
세상의 재미있는(일들은 모두 밤에 일어난다)	(밤에 일들은), (밤에 모두), (밤에 일어난다)
세상의 재미있는 일들은(모두 밤에 일어난다)	(일어난다 모두), (일어난다 밤에)

그림 6.6 Skip-gram의 학습 데이터 구성

- Skip-gram의 모델구조



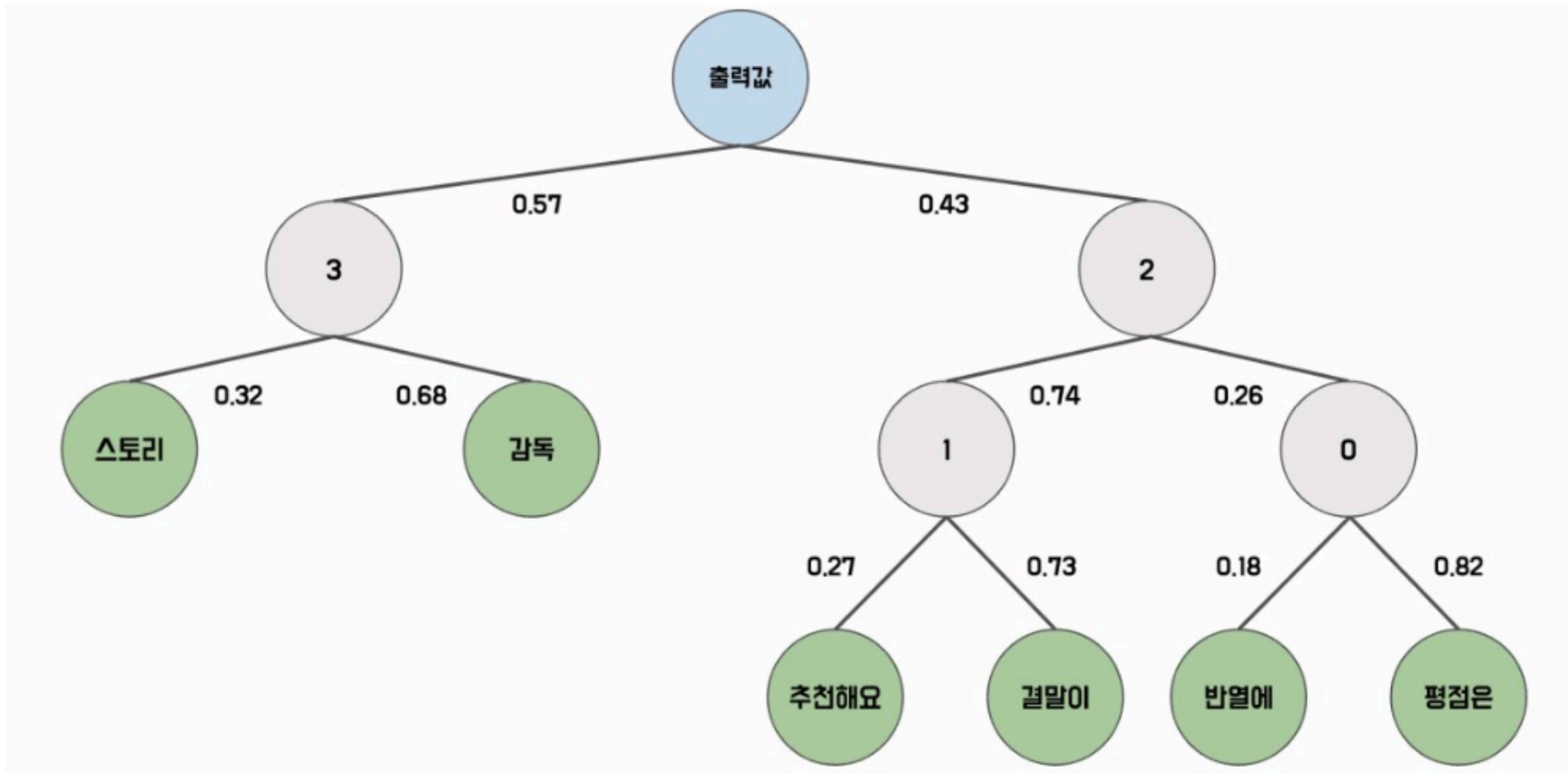
플로우 : 입력 단어의 원-핫 벡터를 투사층에 입력 → 해당 단어의 임베딩 벡터 get → 해당 단어의 임베딩과 가중치 행렬($E \times V$)을 곱해 V 크기의 벡터 get → 해당 벡터에 소프트맥스 연산을 취해 주변 단어 예측(모든 단어를 대상으로 내적 연산 수행)

but 소프트맥스 연산의 이러한 특징으로 말뭉치 크기가 커지면 학습 속도가 느려짐 → 계층적 소프트맥스, 네거티브 샘플링 기법 적용

계층적 소프트맥스

: 출력층을 이진 트리 구조로 표현해 연산을 수행

: 위로 갈수록 자주 등장함!



- 각 노드 : 학습이 가능한 벡터를 가짐 >> 입력값은 해당 노드의 벡터와 내적값을 계산 >> 시그모이드 함수를 통해 확률 계산
- 시간 복잡도 : $O(\log_2 V)$ (V: 단어의 크기)

네거티브 샘플링

: 전체 단어 집합에서 일부 단어를 샘플링하여 오답 단어로 사용 → 문맥에 등장하지 않는 임의의 단어를 샘플링하여 정답 단어와 소프트맥스 연산을 같이 수행

- how?
 - 각 단어의 출현 빈도수 $f(w_i)$
 - 단어가 네거티브 샘플로 추출될 확률 $P(w_i)$
 - 출현 빈도수에 0.75(실험으로 얻어진 정규화 상수)를 제곱

수식 6.9 네거티브 샘플링의 추출 확률

$$P(w_i) = \frac{f(w_i)^{0.75}}{\sum_{j=0}^V f(w_j)^{0.75}}$$

>> 데이터에서 추출된 단어쌍에는 레이블 1, 네거티브 샘플링으로 추출된 단어쌍에는 레이블 0을 부여해 **이진 분류 문제**로 푼다

>> 로지스틱 회귀 모델

입력 데이터	출력 데이터
재미있는	세상의
재미있는	일들은
재미있는	모두

→

입력 데이터	출력 데이터
재미있는. 세상의	1
재미있는. 일들은	1
재미있는. 모두	1
재미있는. 걸어가다	0
재미있는. 주십시오	0
재미있는. 미리	0
재미있는. 안녕하세요	0

그림 6.9 네거티브 샘플링 모델의 훈련 데이터

→ 레이블로 내적 연산 수행하고 이를 시그모이드 함수를 통해 확률값으로 변환(1이면 해당 확률값 up 되도록 반대의 경우 down 되도록 가중치 최적화)

모델 실습 : Skip-gram

0. 임베딩 클래스

: 가중치 행렬의 룩업 연산을 구현하기 쉽도록

: 이산 변수를 연속적인 벡터(모델이 학습하는 동안 단어의 의미와 관련된 정보를 포착& 단어의 유사도 계산)로 변환하는 과정

```
embedding = torch.nn. Embedding(
    num_embeddings,
    embedding_dim,
    padding_idx=None,
    max_norm=None, # 임베딩 벡터의 최대 크기
    norm_type=2.0 #임베딩 벡터의 크기를 제한하는 방법 선택, 1과 2(L1정규화, L2정규화)
)
```

- 임베딩의 유사도 측정하는 방법 : 코사인 유사도

수식 6.10 코사인 유사도

$$\text{cosine similarity}(a,b) = \frac{a \cdot b}{\|a\| \|b\|}$$

모델 실습 : Gensim

: 비교적 간단한 구조의 Word2Vec 모델

: 계층적 소프트 맥스나 네거티브 샘플링 기법을 활용할 때 더 효율적

Gensim 모델

```
from gensim.models import Word2Vec

model = Word2Vec(
```

```

sentences=tokens,      # 학습된 문장들의 리스트
vector_size=128,      # 학습할 임베딩 벡터 크기
window=5,
min_count=1,          # 최소 단어 빈도(이 정도면 무시됨)
sg=1,                 # 1: Skip-gram, 0: CBOW
hs=0,                 # 1: 계층적 소프트맥스 사용, 0 : X
negative=5,           # 네거티브 샘플링 단어 수
epochs=3,
max_final_vocab=10000, # 단어 사전의 최대 크기
workers=3             # 병렬 학습 스레드 수
)

```

- 대용량 텍스트 데이터 처리를 위한 메모리 효율적인 방법 제공
- 학습된 모델 저장하여 관리 가능
- 유사도와 관련된 기능 제공
 - `.most_similar(w, topn)` : 주어진 단어 w와 유사한 단어 top n개
 - `.similarity(w1,w2)` : 두 단어 간의 유사도 계산

but

- 분포 가설 기반으로 학습하므로 단어의 형태학적 특징은 반영 어려움
- 한국어처럼 교착어 특성을 가진 언어에서는 구조적 특징을 제대로 학습하기 어려워(어근, 접사, 조사 규칙) OOV(사전 미등장) 단어가 많아질 수 있음

fastText

fastText란

: 텍스트 분류 및 텍스트 마이닝을 위한 알고리즘

- Word2Vec 와의 차이점
 - 하위 단어 고려 : 나뉘지지 않은 토큰도 N-gram으로 포함
- how to

→ 하위 단어를 고려하여 벡터화

→ 어휘 단위(토큰)를 N-gram으로 하위 단어 집합으로 분해 ex) 서울특별시 ex) 서울, 울특, 특별, 별시(바이그램)

→ 각 하위 단어는 임베딩벡터값을 가진다

→ 임베딩 벡터를 합산하여 입력 단어의 최종 임베딩 벡터를 계산

입력 단어	〈 〉 더하기	N-gram 분해	전체 단어 추가
서울특별시	〈서울특별시〉	〈서울 서울특 울특별 특별시 별시〉	〈서울 서울특 울특별 특별시 별시〉 〈서울특별시〉

1. 토큰의 양 끝에 '〈'와 '〉'를 붙여 토큰의 시작과 끝을 인식할 수 있게 한다.
2. 분해된 토큰은 N-gram을 사용하여 하위 단어 집합으로 분해한다. 트라이그램을 사용한다면, '서울특별시'는 [〈서울, 서울특, 울특별, 특별시, 별시〉]로 분해된다.
3. 분해된 하위 단어 집합에는 나뉘지지 않은 토큰 자체도 포함한다. 이렇게 하위 단어 집합이 만들어지면, 각 하위 단어는 고유한 벡터값을 갖는다.

장점

- 같은 하위 단어를 공유하는 단어끼리 정보 공유해 학습 $\rightarrow$ 비슷한 단어 비슷한 임베딩 벡터 $\rightarrow$ 단어 유사도 up 가능
- OOV 단어도 하위 단어로 나누어 임베딩 계산 $\rightarrow$ ex) '개인택시', '정보처리기사', '임대차보호법' 단어의 하위 단어로 '개인정보보호법'이라는 단어의 임베딩 연산도 $\rightarrow$

모델 실습

fastText 모델

- Word2Vec와 같이 두 단어를 고정 길이 벡터 형태로 표현하기 위해 분산 표현 학습을 수행 & 주변 단어의 정보를 활용하여 단어의 의미 파악 $\rightarrow$ 유사도와 공간적 임베딩의 관계
- CBoW으로 구성 Skip-gram / 네거티브 샘플링 기법으로 학습 / 하위 단어로 학습(<>Word2Vec:기본 단위로 모델 학습)

#fastText 클래스

```
fastText = gensim.models.FastText(
    sentences=tokens, # 학습할 문장: 토큰 리스트들의 리스트
    vector_size=128, # 출력 임베딩 벡터 차원
    window=5,
    min_count=5,
    sg=1, # 1: Skip-gram, 0: CBOW
    epochs=3,
    min_n=2, # n-gram의 최솟값
    max_n=6 # n-gram의 최댓값
)
```

- KorNLI : 자연어 추론을 위한 데이터셋 $\rightarrow$ 두 문장 간의 관계(함의, 중립, 불일치) 분류
 - get_all_texts() : 모든 문장을 튜플 형태로 반환
 - get_all_pairs() : 입력 문장과 쌍을 이루는 대응 문장을 튜플 형태로 반환
- oov 처리 : 하위 단어로 단어 사전 리스트에 없는 단어의 임베딩 계산 $\rightarrow$

```
# oov 처리
oov_token = "사랑해요"
oov_vector = fastText.wv[oov_token]

print(oov_token in fastText.wv.index_to_key)
print(fastText.wv.most_similar(oov_vector, topn=5))
```

순환 신경망

순환 신경망

1. 순환 신경망과 자연어의 연속형 데이터적 특성

: 연속적인 데이터를 처리하기 위해 개발된 인공 신경망의 한 종류

$\rightarrow$ 모델 내부의 과거 상태가 현재 상태를 예측하는 데 사용

- 각 시점이 데이터가 이전 시점의 데이터와 독립적이지 않다는 특성!

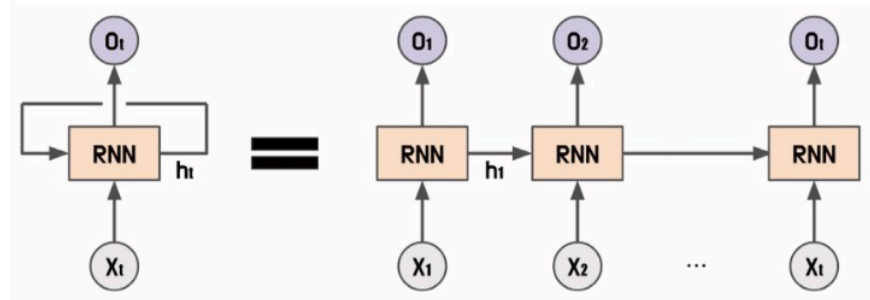
ex) 시계열 데이터, 자연어 문장, 음성처리

- 금요일이 지나면 _
- 수요일이 지나면 목요일이고, 목요일이 지나면 금요일, 금요일이 지나면 _

>> 자연어는 연속형 데이터의 특성을 갖는다.

>> 문장이 길수록 앞선 단어들과 뒤따르는 단어들 사이에 강한 상관관계 존재

2. 셀구조 (단순 순환 구조)



- 연속형 데이터를 순서대로 받아 처리하며 각 시점마다 은닉 상태의 형태로 저장

수식 6.11 순환 신경망의 은닉 상태

$$h_t = \sigma_h(h_{t-1}, x_t)$$

$$h_t = \sigma_h(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

- $\sigma_h(h)$: 은닉 상태를 계산하기 위한 활성화 함수
- 시점 t에 t-1의 은닉상태 h_{t-1} 과 입력값 x_t 를 받아 현 시점의 은닉상태 h_t 를 계산
- W_{hh} : 이전 상태에 대한 가중치 / W_{xh} : 현재 입력에 대한 가중치

- 출력값 계산

수식 6.12 순환 신경망의 출력값

$$y_t = \sigma_y(h_t)$$

$$y_t = \sigma_y(W_{hy}h_t + b_y)$$

- 순환 신경망의 셀 : 각 시점의 데이터를 입력 받아 은닉 상태와 출력값을 계산하는 노드

3. 일대다 구조(순환 신경망의 다양한 구조)

: 하나의 입력 시퀀스로부터 여러 개의 출력을 생성하는 순환 신경망 구조

ex) 문장(입력) 속 단어의 품사(출력)를 예측하는 작업

- 출력 시퀀스의 길이를 미리 알아야함 → 출력 시퀀스의 길이를 예측하는 모델 함께 구현 필요

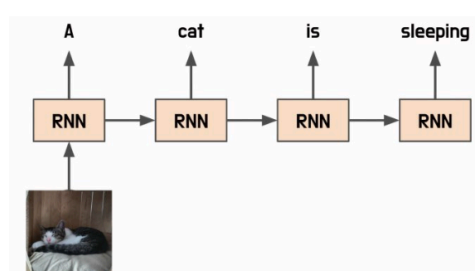


그림 6.13 순환 신경망의 일대다 구조

4. 다대일 구조

: 여러 개의 입력 시퀀스에 대해 하나의 출력값을 생성하는 순환 신경망 구조

ex) 감성 분류 : 특정 문장에서 감정을 예측, 자연어 추론

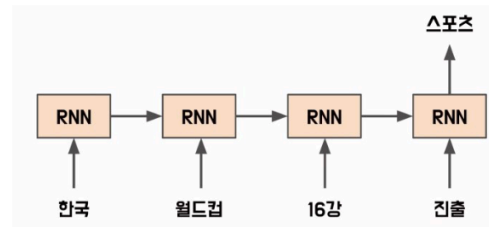


그림 6.14 순환 신경망의 다대일 구조

5. 다대다 구조

: 입력 시퀀스와 출력 시퀀스의 길이가 여러 개인 경우에 사용하는 순환 신경망 구조 but 입력 시퀀스와 출력 시퀀스의 길이를 맞추어야함 with 패딩, 잘라내기

ex) 번역기, 음성 인식기

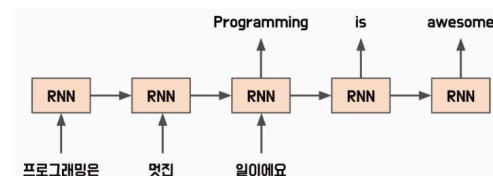


그림 6.15 순환 신경망의 다대다 구조

- 시퀀스-시퀀스 구조 : 인코더(입력 시퀀스 처리 → 고정 크기의 벡터 출력) + 디코더(벡터 → 출력 시퀀스 처리)

6. 양방향 순환 신경망 BiRNN

: 기본 순환 신경망 + 시간 방향이 양방향 → 과거(이전 시점) 정보 & 미래(이후 시점) 정보도 활용

- t시점 이후의 데이터도 t시점의 데이터 예측 가능 >> 역방향 가능

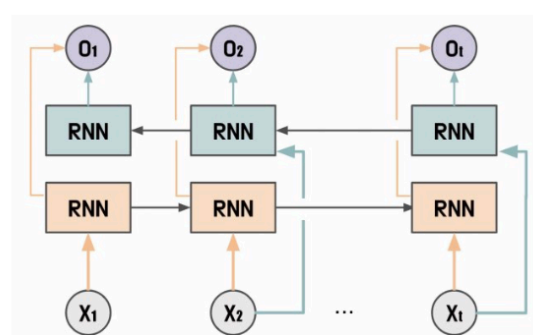


그림 6.16 양방향 순환 신경망 구조

7. 다중 순환 신경망

: 여러 개의 순환 신경망을 층으로 쌓아 구성한 모델로, 각 층의 출력을 다음 층의 입력으로 전달되며 순차 데이터를 처리

but 학습 시간 이슈와 기울기 소실 문제 가능성이 높다

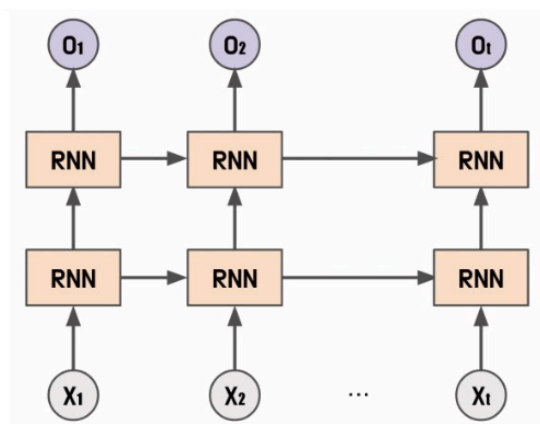


그림 6.17 다중 순환 신경망 구조

순환 신경망 클래스

```
rnn = nn.RNN(
    input_size,      # 입력 특성 x의 벡터 크기
    hidden_size,     # 은닉 상태 h의 벡터의 크기
    num_layers=1,    # RNN의 층수, 2 이상이면 다중 순환 신경망
    nonlinearity="tanh", # 은닉 상태 활성화 함수: tanh or relu
    bias=False,      # bias 사용 여부
    batch_first=True, # 입력 배치 크기를 첫 번째 자원으로 사용 할말
    dropout=0,       # 드롭아웃 확률 for 과대적합 방지 btw 다중 레이어 RNN
    bidirectional=False # 양방향 RNN 사용 할말
)
```

장단기 메모리 LSTM

: 기존 RNN의 기억력 부족과 기울기 소실 문제를 해결한 모델

- 장기 의존성 문제 : 일반적으로 특정 시점에서 이전 입력 데이터의 정보를 활용하므로 먼 과거는 잘 기억 못함 → 충분히 전달되지도 않음
- 활성화 함수로 사용되는 tanh이나 ReLU로 기울기 소실이나 폭주할수도

아래와 같이 RNN과 비슷한 구조 + 메모리 셀 + 게이트

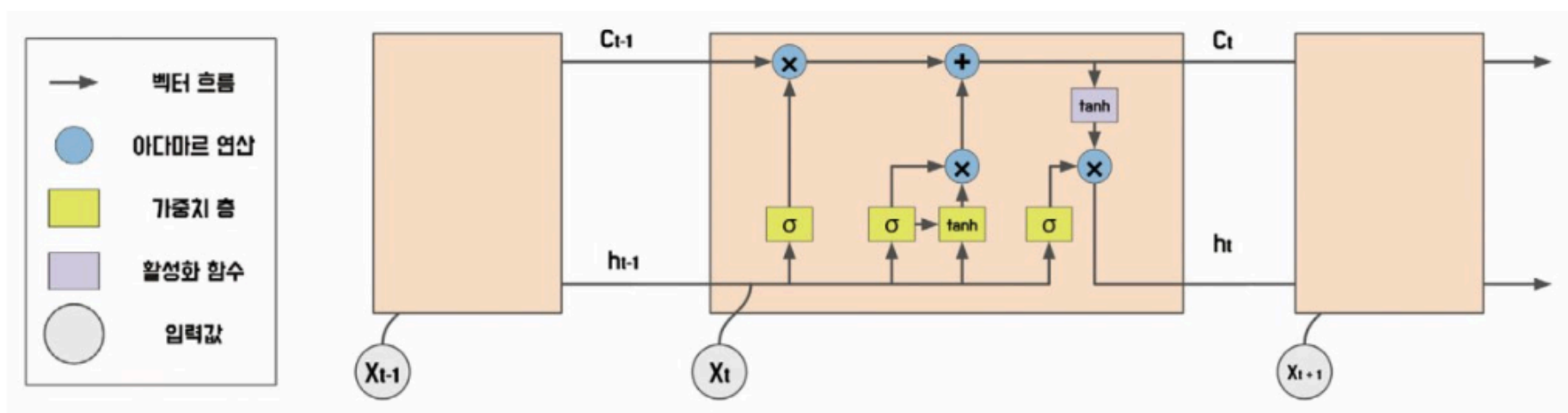


그림 6.18 장단기 메모리 구조

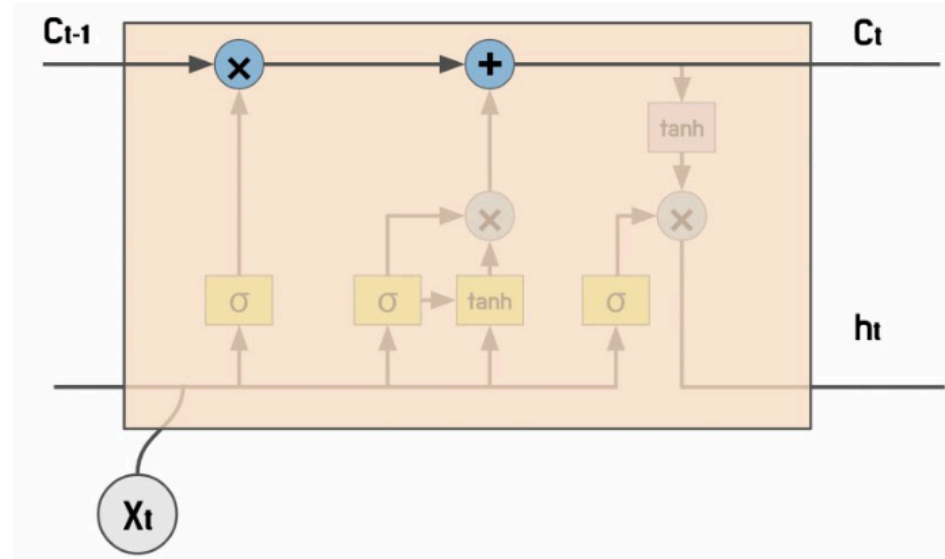
- 구성 요소

(1) 셀 상태 : 정보를 저장하고 유지하는 메모리 역할 → 망각 게이트와 입력 게이트에 의해 제어됨

(2) 망각 게이트 : 현재 입력과 이전 셀 상태를 입력으로 → 장단기 메모리에서 이전 셀 상태에서 어떤 정보를 삭제할지 결정하는 역할

- (3) 기억 게이트(입력 게이트) : 현재 입력과 이전 셀 상태를 입력으로 → 새로운 정보를 어떤 부분에 추가할지 결정
- (4) 출력 게이트 : 현재 입력과 이전 셀 상태, 추가된 정보를 입력으로 → 셀 상태 정보 중 어떤 부분을 출력할지 결정

장단기 메모리 구조



- 메모리셀(<>RNN의 은닉 상태) : 현재와 이전 시점의 은닉 상태를 기반으로 정보를 계산하고 저장 but 출력값 계산에 직접 사용 X

[세가지 게이트는 모두 활성화 함수로 시그모이드를 사용]

1. 망각 게이트

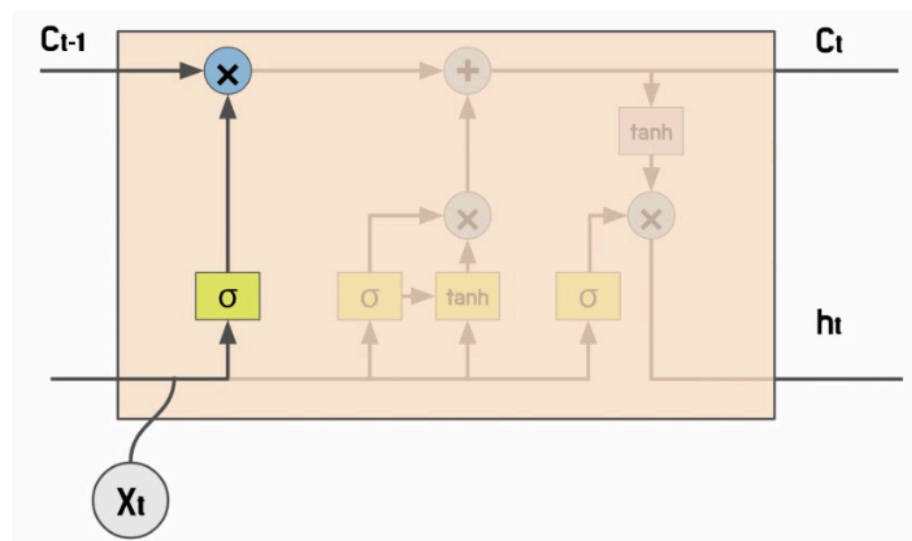


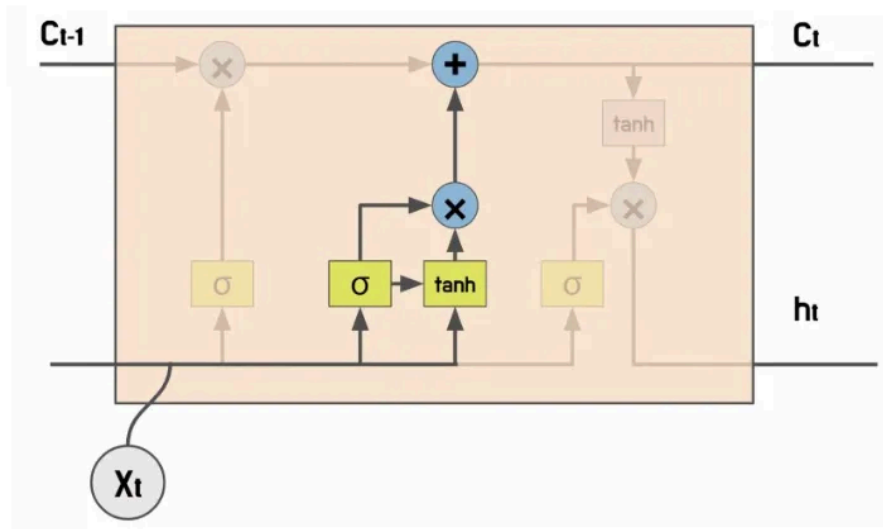
그림 6.20 장단기 메모리의 망각 게이트

- 시그모이드 함수를 거치면 0과 1 사이의 값이 출력되며, 이 값은 어떤 정보를 유지할 것인지, 아니면 망각할 것인지를 결정한다.
- 출력값 1 >> 이전 시점의 기억 상태가 현재 시점의 기억 상태에 완전히 유지, 0 >> 이전 시점이 현재 시점에 전혀 반영 X
- 결과값과 메모리 셀을 곱한 값으로 이전 시점의 메모리 셀을 업데이트한다.

수식 6.13 망각 게이트

$$f_t = \sigma(W_x^{(f)}x_t + W_h^{(f)}h_{t-1} + b^{(f)})$$

2. 기억 게이트



- 현재 입력과 이전 셀 상태를 입력받아 시그모이드 함수를 거친 값과 tanh를 거친 값의 곱으로 새로운 기억값 계산
- $\tanh$: 입력값 $\rightarrow [-1,1]$ 로 변환 $\rightarrow g_i$
- 어떤 정보를 얼마나 추가할지 결정 $\rightarrow i_i$

수식 6.14 기억 게이트

$$g_i = \tanh(W_x^{(g)}x_t + W_h^{(g)}h_{t-1} + b^{(g)})$$

$$i_i = \text{sigmoid}(W_x^{(i)}x_t + W_h^{(i)}h_{t-1} + b^{(i)})$$

- i_i : 새로운 은닉 상태의 기억을 제어, $[0,1]$ 의 값을 가지므로 기억 결정하는 가중치 역할 $\gg 1$ 에 가까울수록 기억할 정보 up

3. 최종 메모리셀 계산

수식 6.15 메모리 셀

$$c_t = f_t \odot c_{t-1} + g_i \odot i_t$$

- 망각 게이트와 기억 게이트의 정보로 현재 시점 메모리 셀 값 계산
 - f_t : 망각 게이트 출력값 $\rightarrow 0$ 에 가까울수록 이전 시점의 영향 less
 - c_{t-1} : 이전 시점의 메모리 셀값

4. 출력 게이트 계산

- 출력 게이트 : 현재 시점의 은닉 상태 제어
- 활성화 함수로 시그모이드 사용

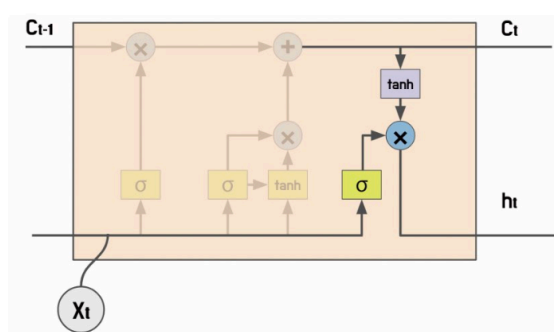


그림 6.22 장단기 메모리의 출력 게이트

수식 6.16 출력 게이트

$$o_t = \sigma(W_x^{(o)}x_t + W_h^{(o)}h_{t-1} + b^{(o)})$$

- 출력 게이트와 현재 메모리 셀 연산 $\rightarrow$ 새로운 은닉 상태 계산 $\rightarrow h_t$

수식 6.17 은닉 상태 갱신

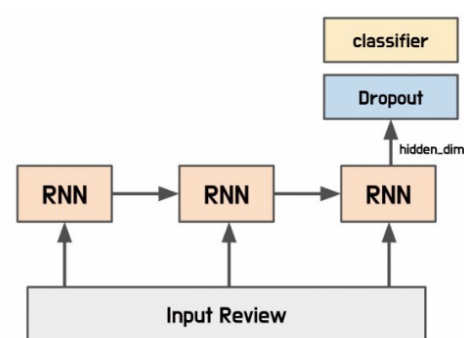
$$h_t = o_t \odot \tanh(c_t)$$

장단기 메모리 클래스

```
lstm = nn.LSTM(  
    input_size,    # 입력 특성 x의 벡터의 크기  
    hidden_size,   # 은닉 상태 h의 크기  
    num_layers=1,  # LSTM 레이어의 층 수  
    bias=False,    # bias 사용 여부  
    batch_first=True,  
    dropout=0,     # 드롭아웃 확률 for 과대적합 방지 btw 다중 lstm  
    bidirectional=False, # 양방향 LSTM 여부 (True면 순방향+역방향 처리)  
    proj_size=0     # 투사 크기(hidden_size보다 작은 값!), 모델의 크기와 복잡도 변경  
)
```

모델 실습

문장 분류 모델 : SentenceClassifier클래스



```
# 문장 분류 모델  
from torch import nn  
  
class SentenceClassifier(nn.Module):  
    def __init__(  
        self,  
        n_vocab, # 단어 사전 크기  
        hidden_dim,  
        embedding_dim,  
        n_layers,  
        dropout=0.5,  
        bidirectional=True,  
        model_type="lstm"  
    ):  
        super().__init__()  
  
        self.embedding = nn.Embedding(  
            num_embeddings=n_vocab,  
            embedding_dim=embedding_dim,  
            padding_idx=0  
        )  
        if model_type == "rnn":  
            self.model = nn.RNN(  
                input_size=embedding_dim,
```

```

        hidden_size=hidden_dim,
        num_layers=n_layers,
        bidirectional=bidirectional,
        dropout=dropout,
        batch_first=True,
    )
elif model_type == "lstm":
    self.model = nn.LSTM(
        input_size=embedding_dim,
        hidden_size=hidden_dim,
        num_layers=n_layers,
        bidirectional=bidirectional,
        dropout=dropout,
        batch_first=True,
    )
if bidirectional:
    self.classifier = nn.Linear(hidden_dim * 2, 1)
else:
    self.classifier = nn.Linear(hidden_dim, 1)
self.dropout = nn.Dropout(dropout)

def forward(self, inputs):
    embeddings = self.embedding(inputs)
    output, _ = self.model(embeddings)
    last_output = output[:, -1, :]
    last_output = self.dropout(last_output)
    logits = self.classifier(last_output)
    return logits

```

- 단어 사전 크기, 순환 신경망 클래스, LTMS 클래스에서 사용하는 매개변수를 입력으로 전달받는다
- model_type으로 rnn인지 LTMS인지 설정
- in 순방향 메서드 : 입력받은 정수 인코딩을 임베딩 계층에 통과시켜 임베딩 값 get → 임베딩 값을 모델에 입력하여 출력값 get → 출력값의 마지막 시점만 활용 → output[:, -1, :]으로 마지막 시점의 결과값만 분리해 분류기 계층에 전달

합성곱 신경망

: 이미지 인식과 같은 CV 분야의 데이터를 분석하기 위해 사용되는 인공 신경망의 한 종류

특징

- 합성곱 연산 : 이미지의 특정 영역에서 입력값의 분포 or 변화량을 계산해 출력 노드 생성
- 지역 특징 : 특정 영역 안에서 합성곱 연산 → 지역 특징을 효과적으로 추출
- 전역 특징 : 여러 합성곱 계층 → 여러 지역 특징을 조합해 전반적인 전역 특징 파악 가능
- 병렬 처리 : 입력 데이터 길이에 관계 X

합성곱 계층

: 입력 데이터에 필터를 합성곱해 데이터의 특징 추출(피쳐맵)

특징

- 모든 위치에서 동일한 필터 사용 → 모델 매개변수를 공유 → 학습해야 할 매개변수 수가 감소 → 과대적합 방지

- 같은 필터 사용 → 어디에 있던 동일한 피쳐맵 추출 가능

1. 필터

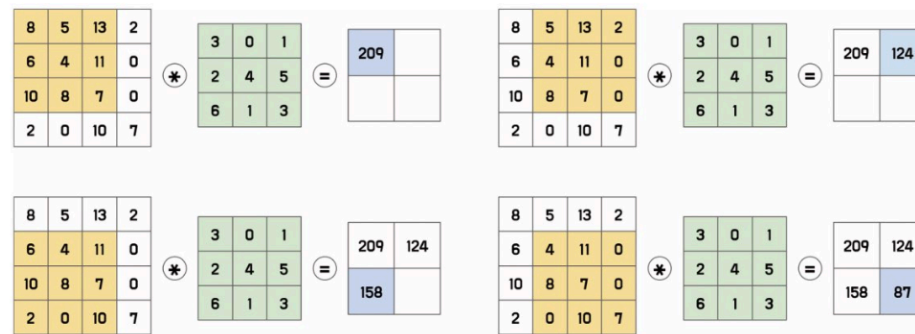


그림 6.24 특징 맵 추출 과정

- 커널 또는 윈도우
- 일정 간격(stride)마다 이동하면서 입력 데이터와 conv 곱을 수행 >> feature map 생성
- 합성곱? 입력 이미지와 커널 내의 매칭되는 부분을 곱한 후 더한 원소

2. 패딩

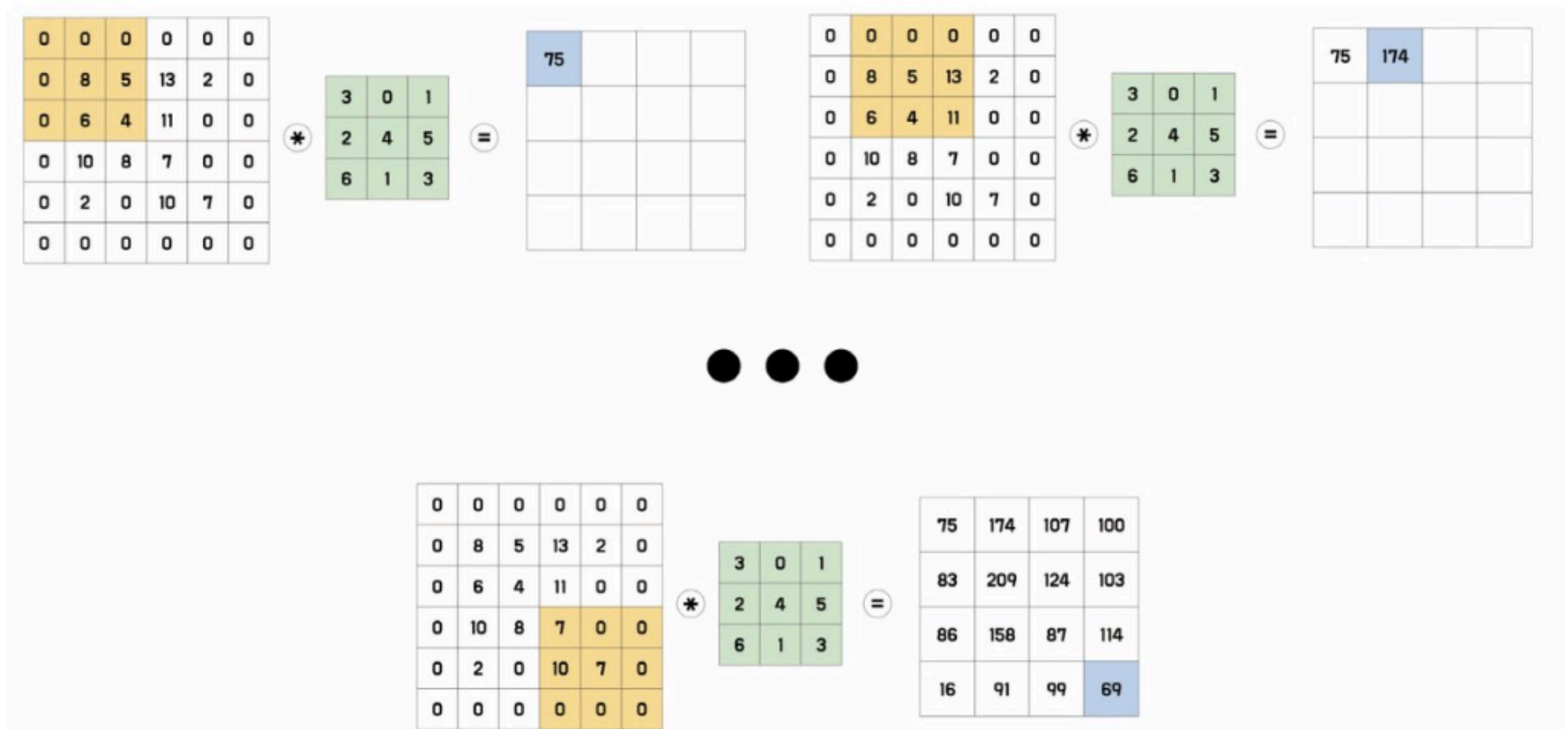


그림 6.25 제로 패딩

- 합성곱 연산 수행하면 feature map의 크기가 작아진다 → 합성곱 신경망 깊게 쌓기에 제약
- 패딩 : 위의 현상을 방지하기 위해 + 작은 크기의 필터로도 정보를 반영하기 위해 입력이미지 등에 특정 값을 덧붙이는 것 ex) 덧붙이는 값이 zero면 zeropadding

3. 간격

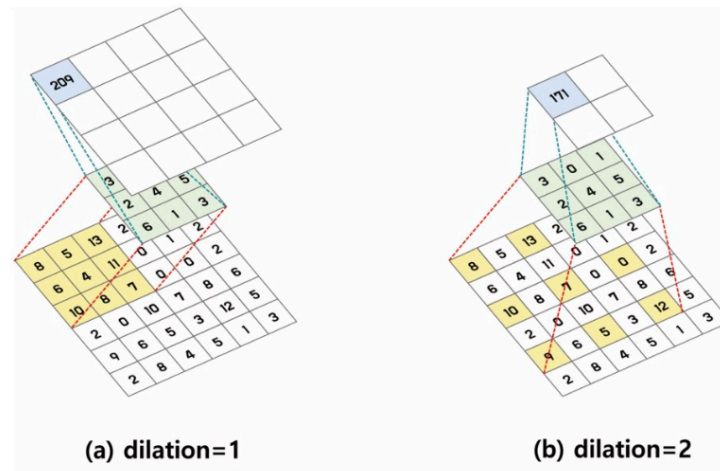
- Stride : 필터가 한 번에 이동하는 크기 ex) Stride=1이면 한 픽셀씩, 2이상이면 여러 칸씩 건너뛰며 연산
- Stride 크기로 뭘함? 출력 데이터 크기 조절 ㄱㄴ
 - Stride=1, 입력 데이터와 동일한 크기의 출력 데이터 생성
 - Stride≥2, 입력 데이터보다 작아진 출.데

- Stride 조정 : 입력 데이터의 공간적 정보 유지(작게) or 감소(크게) ㄱㄴ

4. 채널

- 입력 데이터와 필터 간의 합성곱 연산 → 채널에서 수행
Ex) 3차원 입력(RGB)은 같은 위치의 값들끼리 연산됨 → 공간 정보 유지하면서 feature map 추출 ㄱㄴ
- 채널이 많아지면? 학습하는 특징의 다양성 증가 → 모델의 표현력 up & 더 많은 종류의 특징 학습 → 복잡한 문제 해결 ㄱㄴ
- but 매개변수 증가 이슈 & 과대적합 이슈

5. 팽창



- 팽창 : 합성곱 연산 시 필터와 입력 사이에 간격 not stride를 두어, 더 넓은 영역을 고려하도록 만드는 기법 → dilation이 의미하는게 해당 간격
- 필터의 크기를 키우지 않고 더 넓은 영역 고려 ㄱㄴ → 깊고 복잡한 모델 구성 ㄱㄴ & 매개변수 줄이기 ㄱㄴ
- but 연산량 증가 이슈 & dilation이 너무 커지면 인접 픽셀값 고려 안해서 공간정보 보존 X

합성곱 계층 클래스

2차원 합성곱 계층 클래스

```
conv = torch.nn.Conv2d(
    in_channels,    # 입력 채널 수 ex) RGB는 3
    out_channels,   # 출력 채널 수
    kernel_size,    # 필터 크기
    stride=1,
    padding=0,      # 입력 주변에 덧붙일 0의 개수
    dilation=1,     # 팽창 간격
    groups=1,       # 채널 그룹 수
    bias=True,      # bias 사용 여부
    padding_mode="zeros" # 패딩 방식: "zeros" : 제로패딩, "reflect": 가장자리 반사, "replicate" : 가장자리 값 복제
)
```

- 출력 채널의 크기 계산식

수식 6.18 합성곱 계층 출력 크기

$$L_{out} = \left\lceil \frac{L_{in} + 2 \times padding - dilation \times (kernel_size - 1) - 1}{stride} + 1 \right\rceil$$

활성화 맵

: 합성곱 계층에서 추출한 feature map에 활성화 함수를 적용해 얻은 출력값

→ 활성화 함수(ex. 렐루)를 적용해 비선형성 추가

- 다음 계층의 input 값으로

특징

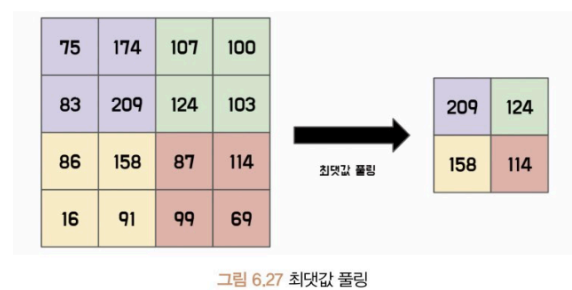
- 비선형성 → 모델이 복잡한 패턴이나 추상적인 특징을 학습할 수 있도록
- 활성화 맵을 시각화 → 모델이 어떤 특징 학습하는지 어떤 부분이 활성화되었는지 이해 가능

풀링

: 특징 맵의 크기를 줄이는 연산, 합성곱 계층 다음에 적용 → 특징 맵의 크기를 줄이고 연산량을 감소&정보 압축

(1) 최댓값 풀링

- 특정 크기의 필터 내 원소값 중 가장 큰 값 선택



↳ 여기에서는 풀링 간격은 2

(2) 평균값 풀링

- 특정 크기의 필터 내 원소값의 평균값 선택

but 입력 데이터의 위치 정보 일부 손실 → 성능 저하

요즘에는 이것보다 CNN Layer의 간격 설정해 입력데이터의 공간적 크기 줄임

1. 풀링 클래스

최댓값 풀링

```
pool_max = nn.MaxPool2d(
    kernel_size=2, # 풀링 커널 크기 2*2
    stride=2,      # 풀링 간격
    padding=0      # 패딩방식 : 제로 패딩
)
```

평균값 풀링

```
pool_avg = nn.AvgPool2d(
    kernel_size=2, # 풀링 커널 크기 2*2
    stride=2,      # 풀링 간격
    padding=0,     # 패딩방식 : 제로 패딩
    count_include_pad=True # 패딩 0도 평균 계산에 포함하는가
)
```

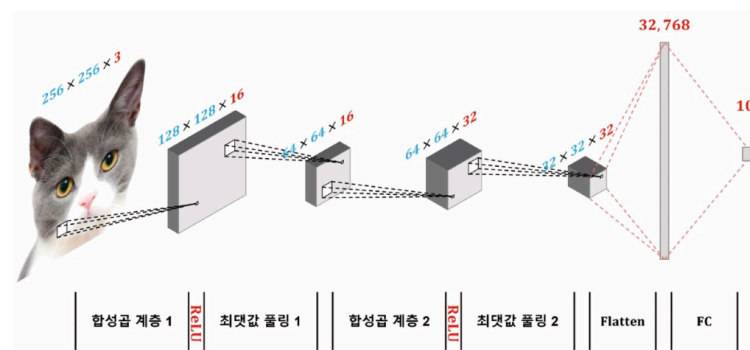
수식 6.20 평균값 풀링 출력 크기

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - kernel_size}{stride} + 1 \right\rfloor$$

완전 연결 계층

: 각 입력 노드가 모든 출력 노드와 연결된 상태 → 입출력 간의 모든 가능한 관계 학습 가능

- 뭘하는데..? 출력 노드의 수 조절 → 1차원 벡터 형태의 결과물

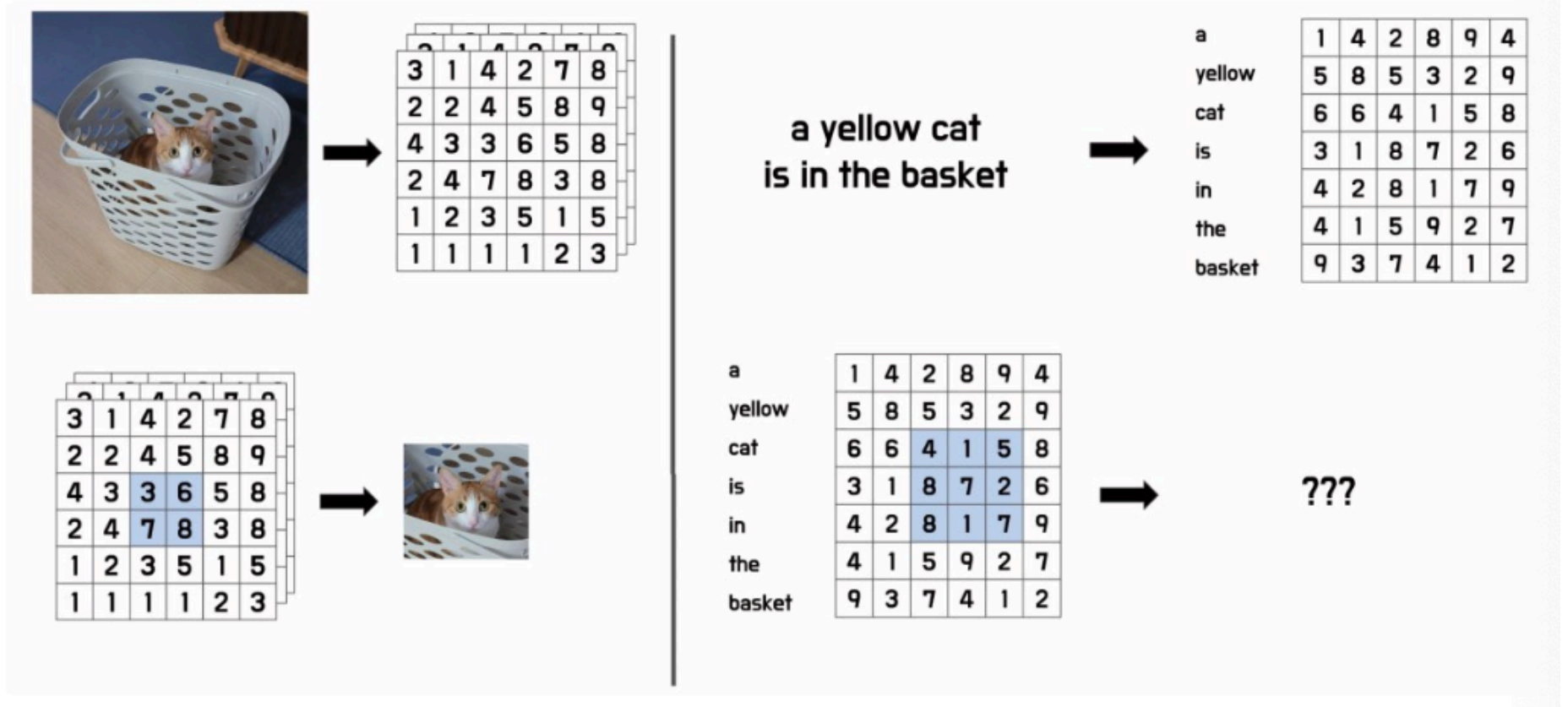


- 입력값 : conv층과 풀링층을 거친 feature map
- 입력값 평탄화 작업해서 1차원 벡터로 flatten
- 출력값 : 전체 입력 feature map과 가중치의 내적 연산 → 추출된 특징에 Softmax나 sigmoid 적용해 분류 모델로

```
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        # Conv+ReLU+Pool *2
        self.conv1 = nn.Sequential(
            nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, stride=2, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )
        self.conv2 = nn.Sequential(
            nn.Conv2d(in_channels=16, out_channels=32, kernel_size=3, stride=1, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )
        # 마지막 feature map (c=32 h=8 w=8)
        self.fc = nn.Linear(32 * 8 * 8, 10) # 10개 클래스로

    def forward(self, x):
        x = self.conv1(x)
        x = self.conv2(x)
        x = x.flatten(start_dim=1)
        x = self.fc(x)
        return x
```

모델 실↪습



↳ 이미지와 다르게 텍스트 데이터(1차원 벡터)는 1차원 합성곱 진행

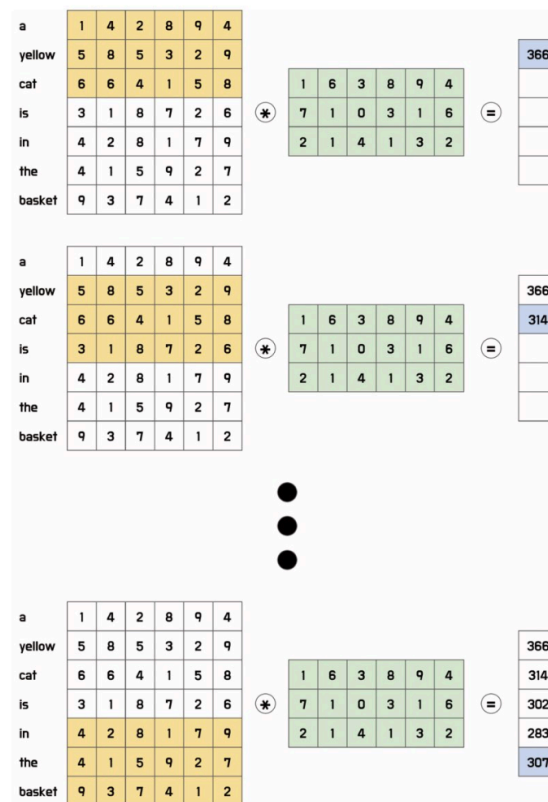


그림 6.30 텍스트 임베딩 입력값의 1차원 합성곱

↳ 텍스트 임베딩 입력값에 크기가 3인 1차원 합성곱 연산을 stride 1로 이동하면서 수행

↳ 필터 크기가 3? 인접한 3개의 토큰에 대해 연산 수행 like n-gram

>> 1차원 합성곱 : 1차원 벡터를 출력값으로 → 풀링하면 하나의 스칼라 값

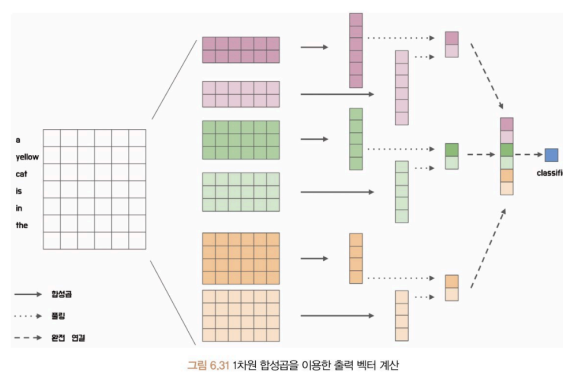


그림 6.31 1차원 합성곱을 이용한 출력 벡터 계산